

NUFFTcf: FAST AUTO- AND CROSS-CORRELATION FUNCTION ESTIMATION FOR IRREGULARLY-SAMPLED TIME SERIES VIA THE NON-UNIFORM FFT

JEAN-ERIC CAMPAGNE 

Université Paris-Saclay, CNRS/IN2P3, IJCLab, 91405 Orsay, France

Version July 26, 2026

Abstract

Estimating the auto-correlation function (ACF) of a single, and the cross-correlation function (CCF) between two, irregularly-sampled scalar time series is a standard task across different physics fields as astronomy, hydrogeology, and environmental fields. We aim to provide an ACF/CCF estimator that is numerically consistent with established, well-validated kernel-weighted definitions, while scaling as $O(n \log n)$ rather than $O(n^2)$.

nufftcf evaluates the Wiener–Khinchin theorem for irregularly sampled data using the Non-Uniform Fast Fourier Transform (NUFFT), via the Flatiron Institute’s **FINUFFT** library, and using the Gaussian- and rectangular-kernel estimators. An $O(n)$ two-pointer scan replaces the naive $O(n^2)$ computation of the effective pair count that normalizes each lag bin. Direct, real-space implementations of the same estimators, and dedicated classical-FFT estimators for regularly-sampled data, are provided alongside the NUFFT route as accuracy references, all sharing a single calling convention.

We validate **nufftcf** against **pastas** (regular-grid ACF, to numerical precision) known in groundwater time series analysis and against **pyZDCF** (CCF, on cases with known theoretical cross-correlation) known in astronomical time series analysis. We measure a speed-up of approximately [XX]x over **pastas** for ACF estimation at $n = [XX]$, with the NUFFT estimator’s residual spectral-leakage bias quantified at [XX]% for strongly periodic, gappy input. **[TODO: summarize validation and benchmark results here.]**

Subject headings: Time series analysis, Correlation function, Non-uniform Fast Fourier Transform

1. INTRODUCTION

Many measurements of physical, environmental, and astrophysical systems take the form of a scalar time series that is not regularly sampled in time. Ground-based photometric monitoring of active galactic nuclei (AGN) is interrupted by weather, daylight, and telescope scheduling; groundwater levels are logged during irregular field visits; paleoclimate proxies are deposited at a rate that itself varies with the climate signal being measured. A recurring analysis task for such data is to estimate the autocorrelation function (ACF) of a single series, or the cross-correlation function (CCF) between two independently and irregularly sampled series, at a set of time lags τ . Applications range from measuring a characteristic variability or persistence timescale, to detecting a time delay between two related signals. For instance in AGN reverberation mapping, where the CCF between a driving continuum light curve and a reprocessed emission-line light curve encodes the size of the emitting region (Welsh 2024; Jankov et al. 2022a; Sánchez-Sáez et al. 2021; Shapovalova et al. 2019; Kovačević et al. 2014), one uses the discrete correlation function (Edelson & Krolik 1988) and also for instance the python version (**pyZDCF**) of the code by Alexander (1997).¹

Because the classical, textbook estimator of the ACF/CCF assumes a common, regular sampling grid (e.g. Stoffer 2026), irregular sampling has historically been handled with different technics (see Kreutzer et al. 2023; Rehfeld et al. 2011): resampling the data onto a

regular grid by interpolation before applying the standard Fourier-based estimator; binning observation pairs by their time-lag separation (“slotting”); weighting observation pairs by a smooth kernel of the time-lag separation rather than binning them; or bypassing the lag domain altogether and working with a Fourier-domain estimator adapted to irregular sampling, such as the Lomb–Scargle periodogram (VanderPlas 2018; Scargle 1982; Lomb 1976).

Each of these approaches has been implemented, refined, and benchmarked by different research communities—astronomy, geoscientific context among them—largely independently of one another (Section 2). What these approaches share is a computational cost that scales at best linearly per output lag and, in the pairwise binning/weighting case, quadratically overall in the number of observations n . This is unproblematic for the short, sparse times series where these methods were originally designed for (typically $n \sim 10^2$ – 10^3), but certainly would become a practical bottleneck for the longer, denser irregular time series increasingly produced by continuous environmental monitoring, high-cadence time-domain surveys, or long instrumental records ($n \sim 10^4$ – 10^6), and for workflows that require many repeated evaluations (parameter sweeps, bootstrap resampling, batch processing of large numbers of light curves or well records).

In this paper we present **nufftcf**, an open-source Python package that computes ACF/CCF estimates by evaluating the Wiener–Khinchin theorem with the Non-Uniform Fast Fourier Transform (NUFFT) and classical-FFT for irregularly- and regularly-sampled time series re-

jean-eric.campagne@ijclab.in2p3.fr

¹ Description of other use-cases in *Pastas* and elsewhere.

spectively. This leads to the cost reduction of the kernel-weighted estimators in widest use (e.g. the Gaussian-kernel estimator used by the `pastas` hydrogeological time-series package) from $O(n^2)$ to $O(n \log n)$. `nufftcf` additionally provides direct, “real-space” implementations of the same estimators as an accuracy reference. All estimator families share a single calling convention, so that a user can trade accuracy against speed without altering downstream analysis code.

The development of `nufftcf` yields the following contributions:

1. We demonstrate that the Gaussian- and rectangular-kernel ACF/CCF estimators—previously established on statistical grounds (Rehfeld et al. 2011)—can be reformulated as a pair of Non-Uniform Fast Fourier Transform (NUFFT) evaluations. This reformulation ensures numerical consistency with existing definitions while achieving an $O(n \log n)$ computational complexity.
2. We introduce an $O(n)$ analytical algorithm, based on a two-pointer scan, to compute the effective pair count for normalizing each lag bin. This replaces the naive $O(n^2)$ all-pairs approach, significantly improving efficiency.
3. We validate `nufftcf` through comparisons with `pastas` (Collenteur et al. 2019) and `pyZDCF` (Jankov et al. 2022b) on synthetic ACF and CCF problems, and quantify the resulting speed-up.

The paper is organized as follows. Section 2 reviews prior approaches to ACF/CCF estimation for irregularly-sampled data and situates `nufftcf` relative to them. Section 3 lays out the methodology, from the regularly-sampled case through the four established families of irregular-sampling estimators to the NUFFT reformulation used by `nufftcf`. Section 4 compares `nufftcf` to `pastas` and `pyZDCF` on ACF and CCF use cases. Section 5 presents timing benchmarks. Section 6 discusses limitations and guidance on estimator choice, and Section 7 concludes.

2. RELATED WORKS

2.1. Slotting and the discrete correlation function

The earliest widely used approach specific to irregular sampling is the *slotting* technique brought into astronomy by Edelson & Krolik (1988) as the discrete correlation function (DCF). Slotting bins the products of pairs of (standardized) observations by their time-lag separation into fixed-width bins, and averages within each bin. It is simple and model-free, but the resulting correlation-function estimate is not guaranteed to be positive semi-definite and necessitate post-processing (Rehfeld et al. 2011). This has led to the use of kernel-weighted techniques (see below)

2.2. Equal-population binning: ZDCF and pyZDCF

Alexander (1997) refined the DCF for the sparse, unevenly sampled light curves typical of AGN monitoring by binning pairs with almost *equal population* per bin

rather than equal width, and by applying a Fisher z -transform to stabilize the variance of the correlation estimate in each bin before computing confidence intervals—the Z-transformed DCF (ZDCF). `pyZDCF` (Jankov et al. 2022b) is the reference Python re-implementation of this algorithm and remains, to our knowledge, the standard tool for CCF estimation in AGN reverberation-mapping studies. Both the DCF’s fixed-width bins and the ZDCF’s equal-population bins require a pairwise scan of the data, giving an $O(n^2)$ cost overall.

2.3. Kernel-weighted estimators

Kernel-weighted estimators, rather than assigning each observation pair to a discrete bin, use a smooth weighting function (*kernel*) of the time-lag separation, allowing pairs to contribute to a lag estimate in proportion to the proximity of their separation to that lag. In this context, Stoica et al. (2009) proposed a sinc kernel, while Bjørnstad & Falck (2001) introduced a Gaussian kernel for this purpose. Notably, the Edelson & Krolik (1988) slotting technique can be interpreted as a rectangular (or box) kernel.

Given the irregular sampling common in geoscientific time series, Rehfeld et al. (2011) conducted a systematic benchmark of kernel-based estimators (Gaussian, sinc, rectangular) against linear interpolation and the Lomb–Scargle approach (below). Using synthetic sinusoidal and autoregressive processes with controlled sampling irregularity, they applied these methods to paleoclimate proxy records. The Gaussian-kernel estimator consistently exhibited the lowest, or close to the lowest, root-mean-square error and bias across nearly all test cases, leading the authors to recommend it over linear interpolation for irregularly sampled data.

This is, to our knowledge, the methodological basis for the ACF estimator implemented in `pastas` (Collenteur et al. 2019), a widely used Python package for groundwater time-series analysis, which offers Gaussian, rectangular, and regular-grid binning options for its ACF. As with slotting, the kernel-weighted estimators implementation evaluate a direct pairwise sum, at $O(n^2)$ overall cost for a full lag range (or $O(n)$ per individual lag).

2.4. Lomb–Scargle-based (Fourier-domain) estimation

A conceptually different route, introduced by Lomb (1976); Scargle (1982) (see also Scargle 1989; VanderPlas 2018), estimates the cross-correlation function by first computing the cross-spectrum via the Lomb–Scargle Fourier transform (LSFT)—an irregular-sampling generalization of the DFT built from a least-squares sinusoid fit—and then applying an inverse Fourier transform to the result, i.e., applying the Wiener–Khinchin theorem (referred to as the Autocorrelation Theorem by Scargle (1989)) in the Fourier domain rather than directly binning or weighting observation pairs in the lag domain (as in slotting or kernel methods). Rehfeld et al. (2011) found this approach competitive with kernel methods for univariate (ACF) problems but markedly worse for bivariate (CCF) problems.

As we will discuss in Section 3.3, `nufftcf` adopts a similar Fourier-domain strategy, sharing Scargle’s approach of estimating correlation functions via Fourier transforms. However, it replaces the Lomb–Scargle transform

(a least-squares fit with a computational cost of $O(n n_f)$ for n_f frequencies) with the Non-Uniform FFT, which achieves a comparable transform in $O(n \log n)$.

2.5. The Non-Uniform FFT and `nufftcf`

The Non-Uniform Fast Fourier Transform (NUFFT) generalizes the classical FFT to handle data sampled at non-uniform points in one domain, mapping them to a uniform grid in the other domain. Unlike a direct non-uniform Fourier sum, which scales as $O(n^2)$, the NUFFT achieves $O(n \log n)$ computational complexity through a three-step process: (1) a *spreading* or *grid-ding* step, where non-uniform data points are convolved with a smooth kernel (e.g., Gaussian, Kaiser-Bessel, or the “exponential of semicircle” kernel introduced by Barnett et al. (2019)) onto a fine, oversampled uniform grid; (2) a standard FFT on this grid; and (3) a *deconvolution* step to correct for the kernel’s effect in the frequency domain. This approach leverages the efficiency of the FFT while maintaining high accuracy, as demonstrated by libraries such as FINUFFT (Barnett et al. 2019), developed at the Flatiron Institute and internally adopted by `nufftcf`.

Modern NUFFT libraries like FINUFFT are *general-purpose numerical tools*, designed for a wide range of applications beyond correlation-function estimation, including medical imaging, radar, and partial differential equation solvers (Barnett et al. 2019). Their flexibility stems from the ability to handle arbitrary non-uniform sampling patterns in 1D, 2D, or 3D, with rigorous error bounds and adaptive kernel choices to balance accuracy and computational cost. For instance, FINUFFT uses an “exponential of semicircle” kernel, which achieves near-optimal exponential convergence rates for aliasing error while being computationally efficient to evaluate (Barnett et al. 2019). Unlike earlier NUFFT implementations (e.g., NFFT (Keiner et al. 2009), CMCL (Greengard & Lee 2004)), which often rely on precomputed kernel tables or are limited to specific use cases, FINUFFT avoids precomputation entirely, reducing memory overhead and enabling efficient and faster on-the-fly evaluations. This makes it particularly well-suited for large-scale or dynamic problems, such as those encountered in time-series analysis.

The closest prior use of a NUFFT-type approach for problems akin to ours is the nonparametric spectral density estimator for irregularly sampled data proposed by Geoga & Beckman (2025), which uses a weighted non-uniform Fourier sum at comparable computational cost. However, this work targets the *power spectral density function* rather than the lag-domain ACF/CCF estimators that are standard in the astronomy and hydrogeology communities and motivate `nufftcf`.

To our knowledge, no existing package—in the astronomy, hydrogeology, or general scientific-Python ecosystems—provides an $O(n \log n)$ NUFFT-based evaluation of the kernel-weighted ACF/CCF estimators established by Rehfeld et al. (2011) and in production use in `pastas`, nor a fast counterpart to the ZDCF/`pyZDCF` family used in AGN reverberation mapping. `nufftcf`’s contribution is deliberately not a new correlation-function *definition*: it targets numerical consistency with the existing, validated Gaussian/rectangular-kernel definitions, while changing *how they are computed*. Using the

FINUFFT library, `nufftcf` achieves the same accuracy of direct computation at a fraction of the computational cost.

3. METHODOLOGY

3.1. Correlation estimation on a regular grid: the Wiener–Khinchin theorem

Let x_t , $t = 0, \dots, n-1$, be a zero-mean, weakly stationary process sampled at N regularly spaced times with sampling interval Δt . The (biased) sample autocovariance at lag $k\Delta t$ is

$$\hat{R}_{xx}(k) = \frac{1}{n} \sum_{t=0}^{n-k-1} x_t x_{t+k}, \quad k = 0, \dots, n-1, \quad (1)$$

and the sample ACF is $\hat{\rho}_{xx}(k) = \hat{R}_{xx}(k)/\hat{R}_{xx}(0)$. The sample cross-covariance and cross-correlation between two co-sampled series x_t and y_t are defined analogously.

For weakly stationary processes, the Wiener–Khinchin theorem states that the autocovariance (or cross-covariance) function and the power (or cross-)spectral density form a Fourier-transform pair. For finite sampled data, this relation is realized numerically through the discrete Fourier transform (DFT): the cross-spectrum is estimated as

$$\hat{S}_{xy}(f) = \hat{X}(f)\hat{Y}^*(f), \quad \hat{R}_{xy}(k) = \mathcal{F}^{-1} \left[\hat{S}_{xy}(f) \right] (k), \quad (2)$$

where $\hat{X}(f) = \mathcal{F}[x](f)$ (and similarly for $\hat{Y}(f)$) denotes the DFT of the sampled series. On a regular grid, this identity is evaluated efficiently using a pair of FFTs—a forward FFT to compute $\hat{X}(f)$ and $\hat{Y}(f)$, a pointwise complex-conjugate product to form $\hat{S}_{xy}(f)$, and an inverse FFT to recover $\hat{R}_{xy}(k)$ —yielding an $O(n \log n)$ algorithm instead of the $O(n^2)$ direct lag-domain summation of Equation 1. This FFT-based approach provides the regular-grid baseline implemented in `nufftcf` (its “classic FFT” estimators, Section 3.4) and motivates the methods for irregular sampling, which either emulate it through resampling, replace the FFT with a non-uniform Fourier transform (e.g., Lomb–Scargle or NUFFT), or abandon the Fourier approach altogether in favor of direct pairwise estimators.

3.2. Four families of estimators for irregularly-sampled data

For an irregularly-sampled series, the sample times $t_1 < t_2 < \dots < t_n$ are not equally spaced, so neither the sum in Equation 1 nor the FFT pair that evaluates it can be applied directly. The literature (Section 2) describes four broad strategies to address this issue.

3.2.1. Linear interpolation onto a regular grid

Resampling x_{t_i} onto an auxiliary regular grid—by linear interpolation or kernel-weighted local interpolation—allows the application of the regular-grid FFT estimator of Section 3.1 to the resampled series. This approach is simple and leverages the efficiency of the FFT machinery. However, interpolation acts as a low-pass filter, introducing a well-documented bias. Specifically, it tends

to produce a positive bias for the autocorrelation function (ACF) and a negative bias for the cross-correlation function (CCF), as demonstrated in the tests of [Rehfeld et al. \(2011\)](#). Furthermore, this bias grows with the irregularity of the sampling and suppresses genuine high-frequency structures in the estimated correlation function.

3.2.2. Slotting: binning in the lag domain

In this approach, every observation pair (x_{t_i}, y_{t_j}) is assigned to the lag bin whose center is nearest to $t_j - t_i$, and the correlation is estimated by averaging within each bin (Section 2.1). A refinement of this method, used by ZDCF/pyZDCF (Section 2.2), adaptively chooses bin boundaries so that each bin contains approximately an equal number of pairs. This adaptation improves the noise properties of the estimate for sparse, unevenly sampled light curves typical of active galactic nucleus (AGN) monitoring. However, it comes at the cost of producing an irregular lag grid in the output. Both variants of this method require an $O(n^2)$ all-pairs scan and do not guarantee that the resulting estimate is positive semi-definite.

3.2.3. Kernel weighting in the lag domain

Instead of using hard bin assignments, this method replaces them with a smooth weight (kernel) $b_k(\cdot)$ applied to the difference between a pair’s time separation and the target lag $k\Delta\tau$ (Section 2.3). The kernel-weighted cross-correlation estimator is defined as:

$$\hat{\rho}_{xy}(k\Delta\tau) = \frac{\sum_{i=1}^{n_x} \sum_{j=1}^{n_y} x_i y_j b_k(d)}{\sum_{i=1}^{n_x} \sum_{j=1}^{n_y} b_k(d)}, \quad (3)$$

where $d = t_j^y - t_i^x - k\Delta\tau$ represents the deviation from the target lag. Typical choices for $b_k(d)$, include the sinc kernel (not implemented in `nufftcf` and `pastas`), the rectangle/box kernel ([Edelson & Krolik 1988](#)):

$$b_k(d) = \begin{cases} 1 & \text{for } |d| \leq h/2, \\ 0 & \text{otherwise.} \end{cases} \quad (4)$$

recognized in fact as the hard bin assignments, and the Gaussian kernel ([Björnstad & Falck 2001](#)), which is the primary choice in `pastas` and `nufftcf`:

$$b_k(d) = \frac{1}{\sqrt{2\pi}h} \exp\left(-\frac{d^2}{2h^2}\right). \quad (5)$$

Here, h is a bandwidth parameter typically set to match the mean sampling interval Δt^{xy} (e.g., $h = \Delta t^{xy}/2$ for the rectangle kernel, $h = \Delta t^{xy}/4$ for the Gaussian kernel; see [Rehfeld et al. 2011](#) for details). The Gaussian kernel is preferred due to its well-documented bias and variance properties (Section 2.3). Evaluating $\hat{\rho}_{xy}$ at all K output lags costs $O(n_x n_y)$ (or $O(n^2 K)$ before recognizing that the shared normalization sum can be reused across lags), as it involves a direct, all-pairs weighted sum.

3.2.4. Fourier-domain estimation via Lomb–Scargle

This approach bypasses the lag domain entirely by computing the Lomb–Scargle Fourier transform of each

series directly from the irregular samples. The (cross-)periodogram is then formed, and an inverse Fourier transform is applied to obtain the (cross-)correlation function via the Wiener–Khinchin theorem ([Scargle 1982](#)). This method reuses the idea of Section 3.1—transform, multiply, invert—but replaces the FFT with a nonuniform transform tailored to irregular sampling, specifically the least-squares Lomb–Scargle fit. However, [Rehfeld et al. \(2011\)](#) found that this method underperforms kernel methods significantly in the bivariate (CCF) case.

`nufftcf` follows a similar high-level strategy as the Lomb–Scargle approach, working in the Fourier domain and invoking Wiener–Khinchin rather than summing pairs directly in the lag domain. However, it replaces the Lomb–Scargle transform with the Non-Uniform FFT. This reformulation reproduces the numerical values of the established kernel estimators (Section 2.3) while achieving their favorable statistical properties at a much lower computational cost. We describe this reformulation in detail next.

3.3. The Non-Uniform Fast Fourier Transform

The NUFFT computes Fourier sums between nonuniformly and uniformly spaced points without forming the $O(n \cdot m)$ direct sum over all n sample points and m output frequencies (or vice versa). Two of its three standard types are relevant here:

- **Type 1** (“nonuniform to uniform”): given values x_j at nonuniform points t_j , $j = 1, \dots, n$, compute $\hat{X}(f_k) = \sum_j x_j e^{-2\pi i f_k t_j}$ at m uniform output frequencies f_k .
- **Type 2** (“uniform to nonuniform”): the adjoint operation, evaluating a uniformly-gridded Fourier series at arbitrary nonuniform output points.

Modern NUFFT algorithms, including FINUFFT ([Barnett et al. 2019](#)) which `nufftcf` uses, compute both types in $O((n + m) \log(1/\varepsilon))$ time for a requested accuracy ε , by (i) *spreading* each nonuniform sample onto a fine uniform grid using a compactly-supported interpolation kernel (FINUFFT uses an “exponential of semicircle” kernel chosen to minimize aliasing error for a given support width), (ii) taking a standard FFT of the fine grid, and (iii) *deconvolving*—dividing by the Fourier transform of the spreading kernel—to correct for the smoothing the kernel introduced. Type 2 reverses this sequence. This spreading kernel is an internal numerical-accuracy device of the NUFFT algorithm itself, distinct from the statistical weighting kernels $b_k(\cdot)$ of Equation 5; we return to how the two relate below.

3.4. From kernel-weighted estimators to NUFFT: the `nufftcf` approach

`nufftcf`’s central observation is that the kernel-weighted estimator of Equation 3 can be evaluated via Wiener–Khinchin using a NUFFT, rather than via a direct pairwise sum, whenever the weighting kernel b_k is itself expressible as (or well approximated by) a smooth, compactly supported function of the lag—which the Gaussian and rectangular kernels used by `pastas` are. Concretely, `nufftcf`:

1. forms the (irregularly sampled) power or cross-spectrum of the input series with a **type-1 NUFFT** onto a uniform frequency grid whose resolution is set by the requested lag range and kernel bandwidth;
2. evaluates the correlation estimate at the requested lags by inverting this spectrum with a **type-2 NUFFT**, applying the Wiener–Khinchin theorem in the same spirit as the regular-grid FFT pair of Section 3.1, but with both legs replaced by their nonuniform-capable counterparts;

so that the total cost of evaluating the correlation function at K lags from n (irregular) samples is $O(n \log n + K \log K)$, instead of the $O(n^2)$ (or $O(nK)$) cost of the direct sum in Equation 3. Because this route implicitly represents the signal on a finite, periodic Fourier basis, it is subject to a small spectral-leakage bias for strongly periodic input with irregular or gappy sampling (quantified in Section 5 and controllable via the number of Fourier modes); `nufftcf` therefore also ships a direct, “real-space” evaluation of the same kernel-weighted estimator, at the original $O(n^2)$ cost, both as an accuracy reference and as the recommended choice when n is modest and this bias must be avoided entirely (Section 6).

Effective pair count. Every estimate in Equation 3 is normalized by $b_k(\cdot)$ summed over all pairs—the *effective number of pairs* contributing to lag k , which both normalizes the correlation estimate and flags under-sampled lags. Computed naively this denominator is itself an $O(n^2)$ all-pairs sum. Because the sample times are sorted, `nufftcf` instead computes it in $O(n)$ with a two-pointer scan (implemented as a `numba`-compiled kernel per supported weighting kernel), and, for the regular-grid classic-FFT estimators, recovers the same quantity for free by filtering the deterministic raw-pair-count ramp with the same discrete filter applied to the correlation numerator. This avoids reintroducing an $O(n^2)$ step alongside an otherwise $O(n \log n)$ estimator.

Unified API. All three estimator families implemented in `nufftcf`—NUFFT-based, real-space, and classic-FFT (regular grid only)—share the calling convention `fn(lags, t, x, ...)` $\rightarrow$ `(c, b)`, so that a

user can move between them, e.g. to spot-check a NUFFT-based result against the real-space reference on a subset of the data, without altering the rest of an analysis pipeline.

4. COMPARISON WITH PASTAS AND PYZDCF

TODO: draft this section from `notebook/pastas_vs_nufftcf.ipynb` and `notebook/zdcf_vs_nufftcf.ipynb` (ACF and CCF use cases, synthetic and/or real data, agreement to what tolerance, any figure(s) to include).

[**TODO:** comparison figures/notebooks — ACF vs `pastas`, CCF vs `pyZDCF`.]

5. TIMING BENCHMARKS

TODO: draft this section from `benchmark/` results (`nufftcf` NUFFT vs `nufftcf` real-space vs `pastas`, ACF timing vs n ; include scaling plot).

[**TODO:** timing benchmark figure(s) and discussion of scaling with n .]

6. DISCUSSION

TODO: discuss (i) when to prefer NUFFT vs real-space vs classic-FFT, (ii) the spectral-leakage bias and how the number of Fourier modes controls it, (iii) guidance for choosing kernel bandwidth h .

7. SUMMARY AND CONCLUSION

TODO.

8. REPRODUCIBLE RESEARCH

The `nufftcf` package is available at <https://github.com/jecampagne/nufftcf> under the **TODO:** license license, together with the notebooks and benchmark scripts used to produce the comparisons and timings in this paper. **TODO:** add PyPI / Zenodo / JOSS DOI once available.

LARGE LANGUAGE MODEL USE DISCLOSURE

[**TODO.**]

ACKNOWLEDGMENTS

We thank the developers of `FINUFFT`, `pastas`, and `pyZDCF`, against which `nufftcf` is validated.

REFERENCES

- Alexander, T. 1997, in *Astrophysics and Space Science Library*, Vol. 218, *Astronomical Time Series*, ed. D. Maoz, A. Sternberg, & E. M. Leibowitz, 163, doi: [10.1007/978-94-015-8941-3_14](https://doi.org/10.1007/978-94-015-8941-3_14)
- Alexander, T. 1997, in *Astronomical Time Series*, ed. D. Maoz, A. Sternberg, & E. M. Leibowitz, Vol. 218 (Springer)
- Barnett, A. H., Magland, J. F., & af Klinteberg, L. 2019, *SIAM Journal on Scientific Computing*, 41, C479, doi: [10.1137/18M120885X](https://doi.org/10.1137/18M120885X)
- Björnstad, O. N., & Falck, W. 2001, *Environmental and Ecological Statistics*, 8, 53, doi: [10.1023/A:1009601932481](https://doi.org/10.1023/A:1009601932481)
- Collenteur, R. A., Bakker, M., Caljé, R., Klop, S. A., & Schaars, F. 2019, *Groundwater*, 57, 877, doi: [10.1111/gwat.12925](https://doi.org/10.1111/gwat.12925)
- Edelson, R. A., & Krolik, J. H. 1988, *ApJ*, 333, 646, doi: [10.1086/166773](https://doi.org/10.1086/166773)
- Edelson, R. A., & Krolik, J. H. 1988, *The Astrophysical Journal*, 333, 646, doi: [10.1086/166773](https://doi.org/10.1086/166773)
- Geoga, C. J., & Beckman, P. G. 2025, *arXiv e-prints*, arXiv:2503.00492, doi: [10.48550/arXiv.2503.00492](https://doi.org/10.48550/arXiv.2503.00492)
- Greengard, L., & Lee, J.-Y. 2004, *SIAM Review*, 46, 443, doi: [10.1137/S003614450343200X](https://doi.org/10.1137/S003614450343200X)
- Jankov, I., Kovačević, A. B., Ilić, D., et al. 2022a, *Astronomische Nachrichten*, 343, e210090, doi: [10.1002/asna.20210090](https://doi.org/10.1002/asna.20210090)
- Jankov, I., Kovačević, A. B., Ilić, D., Sánchez-Sáez, P., & Nikutta, R. 2022b, *pyZDCF: Initial Release*, doi: [10.5281/zenodo.7253034](https://doi.org/10.5281/zenodo.7253034)
- Keiner, J., Kunis, S., & Potts, D. 2009, *ACM Trans. Math. Softw.*, 36, doi: [10.1145/1555386.1555388](https://doi.org/10.1145/1555386.1555388)
- Kovačević, A., Popović, L. Č., Shapovalova, A. I., et al. 2014, *Advances in Space Research*, 54, 1414, doi: [10.1016/j.asr.2014.06.025](https://doi.org/10.1016/j.asr.2014.06.025)
- Kreutzer, L. T., Gillen, E., Briegal, J. T., & Queloz, D. 2023, *MNRAS*, 522, 5049, doi: [10.1093/mnras/stad1223](https://doi.org/10.1093/mnras/stad1223)
- Lomb, N. R. 1976, *Ap&SS*, 39, 447, doi: [10.1007/BF00648343](https://doi.org/10.1007/BF00648343)
- Rehfeld, K., Marwan, N., Heitzig, J., & Kurths, J. 2011, *Nonlinear Processes in Geophysics*, 18, 389, doi: [10.5194/npg-18-389-2011](https://doi.org/10.5194/npg-18-389-2011)
- Sánchez-Sáez, P., Lira, H., Martí, L., et al. 2021, *AJ*, 162, 206, doi: [10.3847/1538-3881/ac1426](https://doi.org/10.3847/1538-3881/ac1426)
- Scargle, J. D. 1982, *ApJ*, 263, 835, doi: [10.1086/160554](https://doi.org/10.1086/160554)
- . 1989, *ApJ*, 343, 874, doi: [10.1086/167757](https://doi.org/10.1086/167757)

Shapovalova, A. I., Popović, L. Č., Afanasiev, V. L., et al. 2019, MNRAS, 485, 4790, doi: [10.1093/mnras/stz692](https://doi.org/10.1093/mnras/stz692)
 Stoffer, D. 2026, astsa: Applied Statistical Time Series Analysis. <https://CRAN.R-project.org/package=astsa>
 Stoica, P., Li, J., & He, H. 2009, Trans. Sig. Proc., 57, 843–858, doi: [10.1109/TSP.2008.2008973](https://doi.org/10.1109/TSP.2008.2008973)

This paper was built using the Open Journal of Astrophysics L^AT_EX template. The OJA is a journal which

VanderPlas, J. T. 2018, ApJS, 236, 16, doi: [10.3847/1538-4365/aab766](https://doi.org/10.3847/1538-4365/aab766)
 Welsh, W. 2024, in AAS/High Energy Astrophysics Division, Vol. 21, AAS High Energy Astrophysics Division Meeting #21, 102.18

provides fast and easy peer review for new papers in the **astro-ph** section of the arXiv, making the reviewing process simpler for authors and referees alike. Learn more at <http://astro.theoj.org>.